

西南交通大學
Southwest Jiaotong University

《软件系统综合课程设计》测试报告

设计题目: 软件系统综合课程设计练习 2 单元测试报告

学　　号: 待填写

姓　　名: 程泽凯

班　　级: 软件工程专业 2023 级

指导教师: 课程指导组

2026 年 6 月 17 日

摘要

这次练习 2 我主要做的是服务层代码的单元测试设计。被测项目仍然是“西南交通大学校园论坛系统”，测试对象选取了本人负责的 3 个代码单元。报告中保留核心代码、控制流图、基本路径、条件-判定覆盖、条件组合覆盖和 JUnit 5 示例，同时补充火车票退票业务的黑盒场景法用例。

关键词：单元测试；白盒测试；基本路径法；条件组合；JUnit；退票场景法

目录

一、 实验过程简述 1

二、 个人测试点总表 1

三、 白盒测试设计与分析 2

3.1 后台封禁用户逻辑 2

3.2 帖子举报创建逻辑 5

3.3 通知已读标记逻辑 10

四、 测试环境与执行记录 14

五、 缺陷记录与回归建议 14

六、 黑盒测试补充：火车票退票场景法 15

6.1 临近发车与办理渠道切换 15

七、 实验收获 17

一、 实验过程简述

我先从项目后端代码中挑出 3 个比较适合做白盒测试的单元，再把每个单元里的判断条件和提前返回分支标出来。随后根据控制流图拆基本路径，用 JUnit 5 和 Mockito 设计对应的测试方法。这个过程里没有把页面点击当成单元测试结果，而是尽量回到服务层方法本身，检查异常类型、返回对象、仓储调用和字段变化。

二、 个人测试点总表

序号	代码单元	方法位置	测试重点与覆盖目标
1	后台封禁用户逻辑（后台用户管理）	AdminOpsService. banUser (Long, String)	重点：最后管理员保护和封禁日志；覆盖目标：保护分支和正常封禁分支都覆盖；方法：基本路径法、条件-判定覆盖、条件组合覆盖
2	帖子举报创建逻辑（内容举报）	ReportService. createPost Report (Long, CreateReport Request, String)	重点：原因合法性、自举报、重复举报；覆盖目标：治理入口的拒绝分支和保存分支都覆盖；方法：基本路径法、条件-判定覆盖、条件组合覆盖
3	通知已读标记逻辑（通知中心）	NotificationService. markRead (Long, String)	重点：通知存在性、归属校验、已读保存；覆盖目标：不泄露他人通知并能标记本人通知；方法：基本路径法、条件-判定覆盖、条件组合覆盖

这 3 个单元的共同点是：代码里都有明确的分支判断，而且每个判断都会影响业务结果。把它们拿来做练习，比只测试简单 getter 或纯数据对象更有意义。

三、 白盒测试设计与分析

本节按照“核心代码段、控制流图、基本路径法、条件-判定覆盖、条件组合覆盖、JUnit 示例”的顺序展开。每个单元内部结构保持一致，但测试说明按各自业务分支单独写。

3.1 后台封禁用户逻辑

被测代码单元：`AdminOpsService.banUser(Long, String)`。该单元包含保留最后一个可用管理员的保护条件，适合做条件组合覆盖。

这段代码属于“后台用户管理”功能，测试时我主要盯住“最后管理员保护和封禁日志”。一开始看代码行数不算多，但异常分支会提前结束流程，如果只测成功路径，很容易把真正容易出错的地方漏掉。

核心代码段摘录：

```
public User banUser(Long userId, String operatorName) {
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new NotFoundException("user not found"));
    if ("ADMIN".equals(user.getRole()) && Boolean.TRUE.equals(user.getEnabled())
            && userRepository.countByRoleAndEnabledTrue("ADMIN") <= 1) {
        throw new ApiException(400, "keep at least one admin");
    }
    user.setEnabled(false);
    User saved = userRepository.save(user);
    User operator = userService.getUserByUsername(operatorName);
    log(operator, "BAN_USER", "USER", userId, "ban user");
    return saved;
}
```

控制流图说明：控制流图里把普通处理语句合并成顺序节点，把会改变执行方向的 if、orElseThrow、短路返回作为判定节点。这样画出来不追求每一行代码一个节点，但能看清测试时必须覆盖的分支。

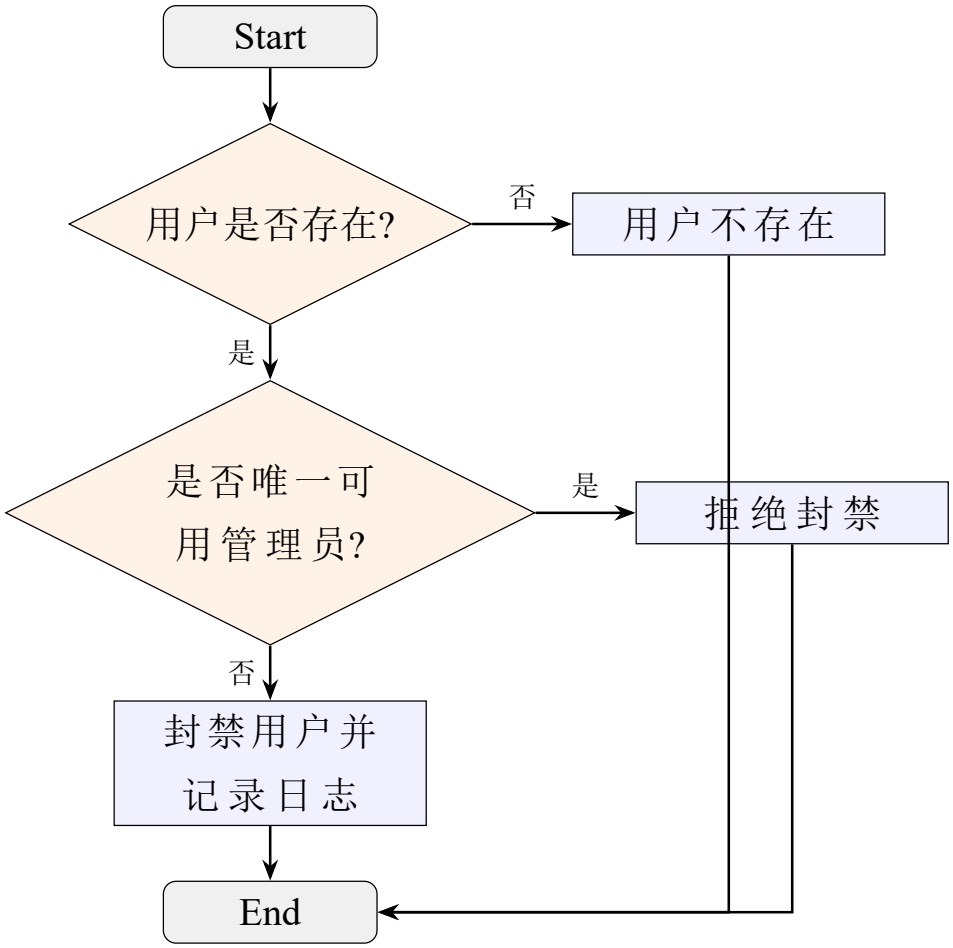


图 **1**　控制流图

基本路径法分析：用户存在和最后管理员保护条件中三个子条件，共规划 5 条路径。本单元的路径设计不是把所有输入都穷举一遍，而是把会让程序提前结束、进入异常处理或完成保存的路线分开。保护分支和正常封禁分支都覆盖。

路径	业务含义
P1	用户不存在。
P2	目标是唯一可用管理员，拒绝封禁。
P3	目标是普通用户，封禁成功。
P4	目标是管理员但仍有其他可用管理员，封禁成功。
P5	目标管理员已禁用，再次封禁保持禁用并记录操作。

这些路径合在一起，能够覆盖主流程和主要异常出口。后续如果代码增加新的 if 分支，需要先回到这里补路径，再补测试方法。

基本路径法测试用例：下表把每条路径落成可执行的 JUnit 用例，重点是让 mock 条件和预期分支一一对应。

编号	测试数据与条件	预期结果
BAN-P1	userId 不存在	NotFoundException
BAN-P2	role=ADMIN, enabled=true, count=1	ApiException 400
BAN-P3	role=USER	enabled=false，写操作日志
BAN-P4	role=ADMIN, enabled=true, count=2	enabled=false
BAN-P5	role=ADMIN, enabled=false	不触发最后管理员保护

这些用例主要用来卡住执行路径。实际写代码时，最容易出问题的是 mock 条件没有配全，导致测试没有走到预想分支。

条件-判定覆盖：本部分关注每个判定条件至少取到一次真值和一次假值。对于短路判断，报告中会把未执行条件单独说明，不把它误写成已经覆盖。

编号	覆盖条件	判定结果
BAN-CD1	isAdmin=T	管理员条件取真
BAN-CD2	enabled=T	启用状态条件取真
BAN-CD3	lastAdmin=T	保护条件取真
BAN-CD4	组合整体为假	允许封禁

条件-判定覆盖的作用是提醒我不要只看返回值，也要看每个 if 的真假两面有没有被触发。

条件组合覆盖：理论组合不一定都可执行，例如前一个判定已经抛异常时，后面的条件不会再执行。因此这里优先列出业务上可行、能被 JUnit 稳定构造的组合。

编号	条件 1	条件 2	条件 3/4	组合期望
BAN-C1	T	T	T	拒绝封禁
BAN-C2	T	T	F	允许封禁
BAN-C3	T	F	-	允许封禁
BAN-C4	F	-	-	允许封禁

组合覆盖比单独的条件覆盖更容易暴露短路逻辑。例如前置条件为真时，后续仓储查询或密码判断可能根本不会发生，测试中应使用 `verify(..., never())` 或异常断言来确认。

JUnit 5 测试类示例：下面是按本单元整理的测试类骨架。导入语句按项目实际包名补齐即可，个别实体构造方法也可以换成项目已有的测试工厂方法。

```
@ExtendWith(MockitoExtension.class)
class AdminBanUserWhiteBoxTest {
    @Mock UserRepository userRepository;
    @Mock OperationLogRepository operationLogRepository;
    @Mock UserService userService;
    @InjectMocks AdminOpsService adminOpsService;

    // 根据实际项目类名补齐 DTO 和实体的构造细节。
    private User user(String name) {
        User user = new User();
        user.setId(1L);
        user.setUsername(name);
        user.setEnabled(true);
        return user;
    }

@Test
void banUserKeepsLastEnabledAdmin() {
    admin.setRole("ADMIN");
    admin.setEnabled(true);
    when(userRepository.countByRoleAndEnabledTrue("ADMIN")).thenReturn(1L);
    assertThrows(ApiException.class, () -> adminOpsService.banUser(admin.getId(), "root"));
}

@Test
void banUserDisablesNormalUserAndLogsOperation() {
    normal.setRole("USER");
    User saved = adminOpsService.banUser(normal.getId(), "root");
    assertThat(saved.getEnabled()).isFalse();
    verify(operationLogRepository).save(any(OperationLog.class));
}
}
```

缺陷与回归关注：未发现缺陷。该保护条件很关键，后续修改用户角色逻辑时应优先回归。这部分没有发现阻塞缺陷，但后续回归时还要注意 DTO 校验、异常提示和前端展示是否与服务层状态保持一致。

3.2 帖子举报创建逻辑

被测代码单元：`ReportService.createPostReport(Long, CreateReportRequest, String)`。该单元防止非法原因、自举报和重复举报，属于后台治理流程入口。

这段代码属于“内容举报”功能，测试时我主要盯住“原因合法性、自举报、重复

举报”。一开始看代码行数不算多，但异常分支会提前结束流程，如果只测成功路径，很容易把真正容易出错的地方漏掉。

核心代码段摘录：

```
public Report createPostReport(Long postId, CreateReportRequest request, String username) {
    User reporter = userService.getUserByUsername(username);
    Post post = postRepository.findById(postId)
            .orElseThrow(() -> new NotFoundException("post not found"));
    String reason = request.getReason().trim();
    if (!ALLOWED_REASONS.contains(reason)) {
        throw new ApiException(400, "bad reason");
    }
    if (post.getAuthor() != null && post.getAuthor().getId().equals(reporter.getId())) {
        throw new ApiException(400, "cannot report yourself");
    }
    if (reportRepository.existsByReporterAndPostAndStatus(reporter, post, STATUS_PENDING)) {
        throw new ApiException(400, "duplicate report");
    }
    Report report = new Report();
    report.setReporter(reporter);
    report.setPost(post);
    report.setReason(reason);
    return reportRepository.save(report);
}
```

控制流图说明：控制流图里把普通处理语句合并成顺序节点，把会改变执行方向的if、orElseThrow、短路返回作为判定节点。这样画出来不追求每一行代码一个节点，但能看清测试时必须覆盖的分支。

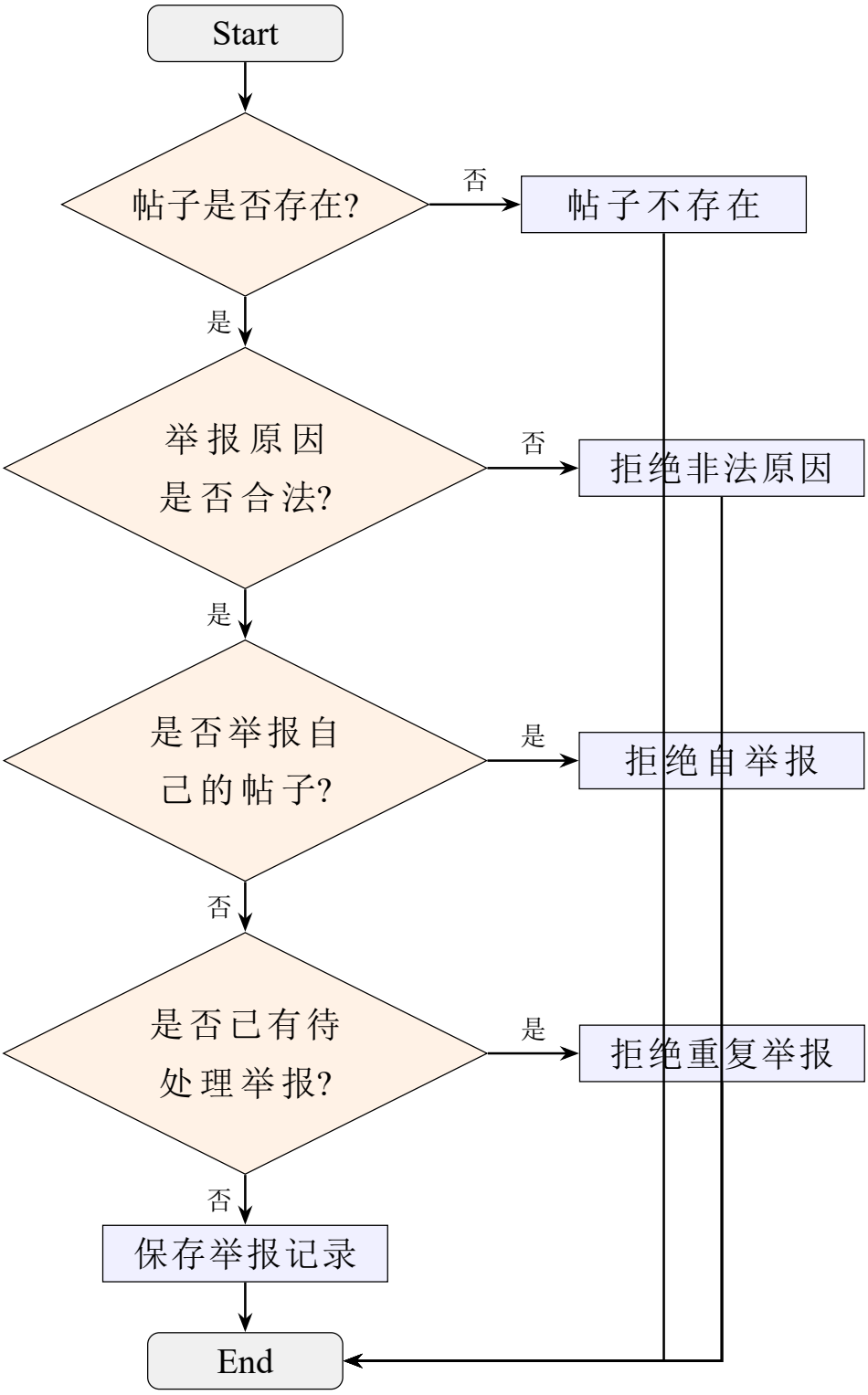


图 **2**　控制流图

基本路径法分析：帖子存在、原因合法、是否举报自己、是否重复四个判定，V(G)=5。本单元的路径设计不是把所有输入都穷举一遍，而是把会让程序提前结束、进入异常处理或完成保存的路线分开。治理入口的拒绝分支和保存分支都覆盖。

路径	业务含义
P1	帖子不存在。
P2	举报原因不在允许集合内。
P3	举报自己的帖子。
P4	同一用户对同一帖子已有待处理举报。
P5	合法举报保存成功。

这些路径合在一起，能够覆盖主流程和主要异常出口。后续如果代码增加新的 if 分支，需要先回到这里补路径，再补测试方法。

基本路径法测试用例：下表把每条路径落成可执行的 JUnit 用例，重点是让 mock 条件和预期分支一一对应。

编号	测试数据与条件	预期结果
REPORT-P1	postId 不存在	NotFoundException
REPORT-P2	reason= 随便写	ApiException 400
REPORT-P3	reporter=post.author	ApiException 400
REPORT-P4	exists pending=true	ApiException 400
REPORT-P5	reason= 垃圾广告，无重复	保存 Report

这些用例主要用来卡住执行路径。实际写代码时，最容易出问题的是 mock 条件没有配全，导致测试没有走到预想分支。

条件-判定覆盖：本部分关注每个判定条件至少取到一次真值和一次假值。对于短路判断，报告中会把未执行条件单独说明，不把它误写成已经覆盖。

编号	覆盖条件	判定结果
REPORT-CD1	postExists=F	帖子不存在
REPORT-CD2	reasonAllowed=F	原因不合法
REPORT-CD3	selfReport=T	拒绝自举报
REPORT-CD4	duplicate=T	拒绝重复举报

编号	覆盖条件	判定结果
REPORT-CD5	全部满足	创建成功

条件-判定覆盖的作用是提醒我不要只看返回值，也要看每个 if 的真假两面有没有被触发。

条件组合覆盖：理论组合不一定都可执行，例如前一个判定已经抛异常时，后面的条件不会再执行。因此这里优先列出业务上可行、能被 JUnit 稳定构造的组合。

编号	条件 1	条件 2	条件 3/4	组合期望
REPORT-C1	F	-	-	-；帖子不存在
REPORT-C2	T	F	-	-；非法原因
REPORT-C3	T	T	T	-；自举报
REPORT-C4	T	T	F	T；重复举报
REPORT-C5	T	T	F	F；成功

组合覆盖比单独的条件覆盖更容易暴露短路逻辑。例如前置条件为真时，后续仓储查询或密码判断可能根本不会发生，测试中应使用 `verify(..., never())` 或异常断言来确认。

JUnit 5 测试类示例：下面是按本单元整理的测试类骨架。导入语句按项目实际包名补齐即可，个别实体构造方法也可以换成项目已有的测试工厂方法。

```
@ExtendWith(MockitoExtension.class)
class ReportCreateWhiteBoxTest {
    @Mock ReportRepository reportRepository;
    @Mock PostRepository postRepository;
    @Mock OperationLogRepository operationLogRepository;
    @Mock ReportReviewLogRepository reportReviewLogRepository;
    @Mock UserService userService;
    @Mock NotificationService notificationService;
    @InjectMocks ReportService reportService;

    // 根据实际项目类名补齐 DTO 和实体的构造细节。
    private User user(String name) {
        User user = new User();
        user.setId(1L);
        user.setUsername(name);
        user.setEnabled(true);
        return user;
    }
```

```
@Test
void createReportRejectsSelfReport() {
    post.setAuthor(reporter);
    request.setReason("垃圾广告");
    assertThrows(ApiException.class,
            () -> reportService.createPostReport(1L, request, "alice"));
}

@Test
void createReportSavesValidReport() {
    request.setReason("虚假信息");
    Report saved = reportService.createPostReport(1L, request, "bob");
    assertThat(saved.getReason()).isEqualTo("虚假信息");
    verify(reportRepository).save(any(Report.class));
}
}
```

缺陷与回归关注：未发现缺陷。建议前端原因下拉框和后端允许集合保持同步，减少非法原因提交。这部分没有发现阻塞缺陷，但后续回归时还要注意 DTO 校验、异常提示和前端展示是否与服务层状态保持一致。

3.3 通知已读标记逻辑

被测代码单元：`NotificationService.markRead(Long, String)`。该单元用 NotFoundException 隐藏他人通知存在性，适合验证权限分支。

这段代码属于“通知中心”功能，测试时我主要盯住“通知存在性、归属校验、已读保存”。一开始看代码行数不算多，但异常分支会提前结束流程，如果只测成功路径，很容易把真正容易出错的地方漏掉。

核心代码段摘录：

```
public Notification markRead(Long id, String username) {
    User recipient = requireUser(username);
    Notification notification = notificationRepository.findById(id)
            .orElseThrow(() -> new NotFoundException("notification not found"));
    if (notification.getRecipient() == null
            || !notification.getRecipient().getId().equals(recipient.getId())) {
        throw new NotFoundException("notification not found");
    }
    notification.setIsRead(true);
    return notificationRepository.save(notification);
}
```

控制流图说明：控制流图里把普通处理语句合并成顺序节点，把会改变执行方向的

if、orElseThrow、短路返回作为判定节点。这样画出来不追求每一行代码一个节点，但能看清测试时必须覆盖的分支。

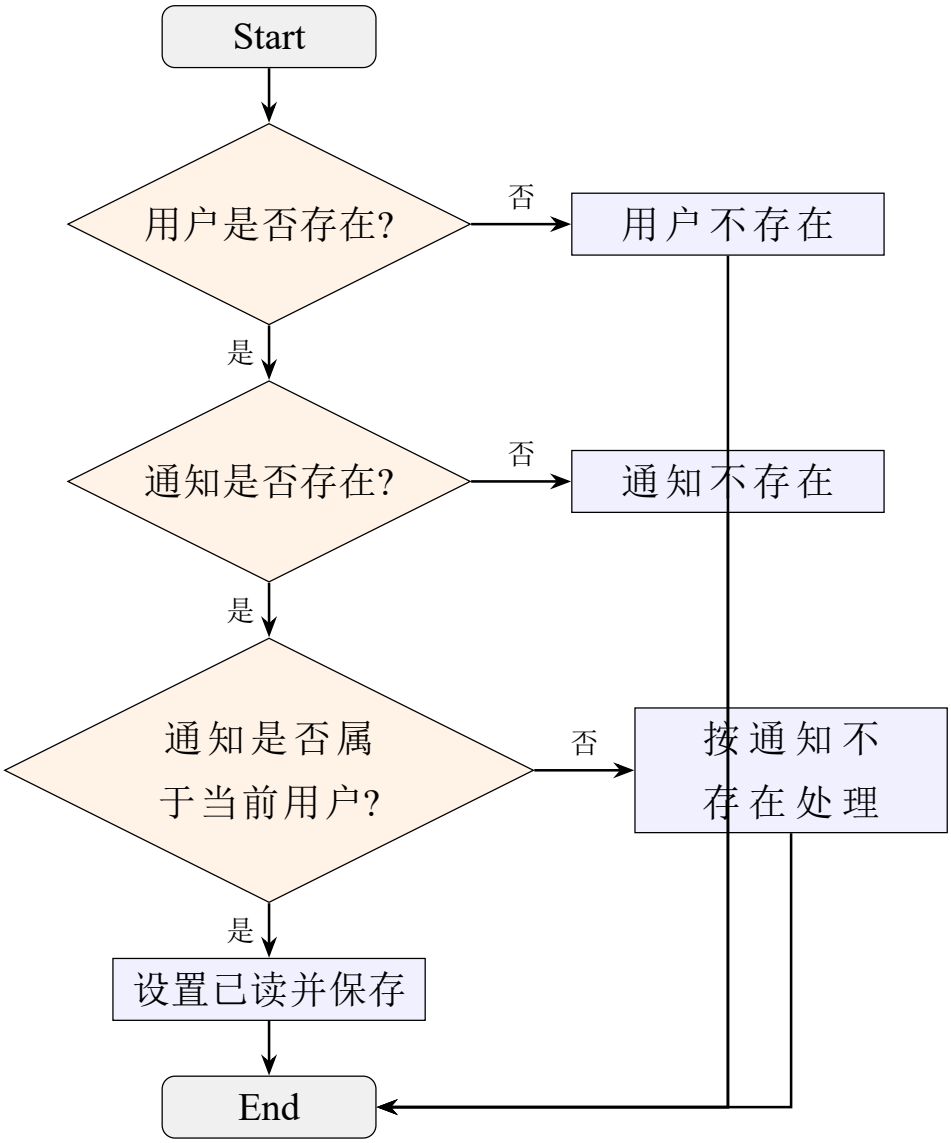


图 **3**　控制流图

基本路径法分析：用户存在、通知存在、通知归属三个判定，V(G)=4。本单元的路径设计不是把所有输入都穷举一遍，而是把会让程序提前结束、进入异常处理或完成保存的路线分开。不泄露他人通知并能标记本人通知。

路径	业务含义
P1	用户名不存在。
P2	通知不存在。
P3	通知存在但不属于当前用户。

路径	业务含义
P4	通知属于当前用户，标记已读成功。

这些路径合在一起，能够覆盖主流程和主要异常出口。后续如果代码增加新的 if 分支，需要先回到这里补路径，再补测试方法。

基本路径法测试用例：下表把每条路径落成可执行的 JUnit 用例，重点是让 mock 条件和预期分支一一对应。

编号	测试数据与条件	预期结果
NOTICE-P1	username 不存在	NotFoundException
NOTICE-P2	notificationId 不存在	NotFoundException
NOTICE-P3	recipientId != currentUserId	NotFoundException
NOTICE-P4	recipientId == currentUserId	isRead=true 并保存

这些用例主要用来卡住执行路径。实际写代码时，最容易出问题的是 mock 条件没有配全，导致测试没有走到预想分支。

条件-判定覆盖：本部分关注每个判定条件至少取到一次真值和一次假值。对于短路判断，报告中会把未执行条件单独说明，不把它误写成已经覆盖。

编号	覆盖条件	判定结果
NOTICE-CD1	userExists=F	用户不存在
NOTICE-CD2	noticeExists=F	通知不存在
NOTICE-CD3	ownerMatch=F	不泄露他人通知
NOTICE-CD4	ownerMatch=T	标记已读

条件-判定覆盖的作用是提醒我不要只看返回值，也要看每个 if 的真假两面有没有被触发。

条件组合覆盖：理论组合不一定都可执行，例如前一个判定已经抛异常时，后面的条件不会再执行。因此这里优先列出业务上可行、能被 JUnit 稳定构造的组合。

编号	条件 1	条件 2	条件 3/4	组合期望
NOTICE-C1	F	-	-	用户不存在
NOTICE-C2	T	F	-	通知不存在
NOTICE-C3	T	T	F	他人通知，仍返回不存在
NOTICE-C4	T	T	T	标记成功

组合覆盖比单独的条件覆盖更容易暴露短路逻辑。例如前置条件为真时，后续仓储查询或密码判断可能根本不会发生，测试中应使用 `verify(..., never())` 或异常断言来确认。

JUnit 5 测试类示例：下面是按本单元整理的测试类骨架。导入语句按项目实际包名补齐即可，个别实体构造方法也可以换成项目已有的测试工厂方法。

```
@ExtendWith(MockitoExtension.class)
class NotificationMarkReadWhiteBoxTest {
    @Mock NotificationRepository notificationRepository;
    @Mock UserRepository userRepository;
    @InjectMocks NotificationService notificationService;

    // 根据实际项目类名补齐 DTO 和实体的构造细节。
    private User user(String name) {
        User user = new User();
        user.setId(1L);
        user.setUsername(name);
        user.setEnabled(true);
        return user;
    }

@Test
void markReadDoesNotExposeOtherUsersNotification() {
    notification.setRecipient(otherUser);
    assertThrows(NotFoundException.class,
            () -> notificationService.markRead(9L, "alice"));
}

@Test
void markReadSetsReadFlag() {
    notification.setRecipient(alice);
    Notification saved = notificationService.markRead(9L, "alice");
    assertThat(saved.getIsRead()).isTrue();
    verify(notificationRepository).save(notification);
}
}
```

缺陷与回归关注：未发现缺陷。该分支体现了权限隐藏策略，需要和接口状态码契约保持一致。这部分没有发现阻塞缺陷，但后续回归时还要注意 DTO 校验、异常提示和前端展示是否与服务层状态保持一致。

四、 测试环境与执行记录

- 操作系统：Windows。
- Java 环境：JDK 17，Spring Boot 3.2.0。
- 单元测试框架：JUnit 5 + Mockito + Maven Surefire。
- 实际执行命令：进入 `jiaodaluntan/backend` 后运行 `mvn test`。
- 本次执行结果：通过 85，失败 0，错误 0，跳过 0。
- 覆盖率工具：本轮未启用 JaCoCo，因此报告只记录 Surefire 的真实单元测试结果，不编造覆盖率百分比。

项目	记录
执行命令	mvn test
执行目录	jiaodaluntan/backend
测试源码位置	src/test/java/com/swjtu/forum
本报告对应测试类	AdminOpsServiceTest（封禁用户）、ReportServiceTest（举报创建）、NotificationServiceTest（已读通知）
测试类数量	18 个
测试方法数量	85 个
成功/失败/错误/跳过	通过 85，失败 0，错误 0，跳过 0
执行时间	12.139 s
完成时间	2026-06-17 16:01:22 +08:00
Surefire 报告	target/surefire-reports，已生成 18 组测试报告
JaCoCo 报告	本轮未启用 JaCoCo；未编造覆盖率百分比
备注	主要验证服务层分支逻辑，页面截图不作为本次单元测试证据

五、 缺陷记录与回归建议

类别	位置	说明
已发现问题	本次 3 个服务层单元	本轮 mvn test 共 85 个测试全部通过，未记录到阻塞级服务层缺陷
潜在风险	输入 DTO 校验	空用户名、非法邮箱、弱密码、空评论等最好在 DTO 或校验器层单独补测试，避免服务层假设输入永远合法
潜在风险	登录限流/时间窗口	限流逻辑依赖时间或缓存状态，建议把 LoginAttemptService 单独写测试，不要只在登录方法里间接覆盖
潜在风险	分页边界	分页工具类建议补 Integer.MAX_VALUE、Integer.MIN_VALUE 等极端值
潜在风险	火车票退票	退票费取整规则容易写错，特别是 2.5 角和 7.5 角附近
回归建议	测试目录	本轮测试已落到真实 src/test/java 目录；后续修改服务层后应继续执行 mvn test

这部分没有为了凑数量去写虚假的 Bug。能确定的问题和风险分开记录，后续回归时更容易判断哪些需要马上修，哪些只是需要加测试保护。

六、 黑盒测试补充：火车票退票场景法

6.1 临近发车与办理渠道切换

我的黑盒测试假设票价为 214.50 元，主线放在临近发车时的渠道切换，同时检查大额票价下手续费进位规则。车票开车时间固定为 2026 年 6 月 30 日 07:52。除特别说明外，场景默认张三未打印纸质车票；为了测试手续费取整规则，最后三条额外设置了不同票价。

基本流：张三进入退票流程，系统读取开车时间、退票申请时间、票价和纸质票状态；先判断办理渠道，再判断手续费比例，最后按 5 角规则取整并给出退款金额。

备选流：如果退票时间正好落在 15 天、48 小时、24 小时或 2 小时边界，需要按需求文字解释边界；如果已打印纸质车票，即使时间满足网上退票，也只能到车站窗口办理。

异常流：不足开车前 2 小时仍从网上入口提交时，系统应提示不能网上办理；手续费取整遇到 2.5 角和 7.5 角附近时，不能简单四舍五入到整数。

```
S0 start
S1 read ticket: departure=2026-06-30 07:52, price, paperTicket
```

```
S2 if before 2 hours and no paper ticket -> online; else station window
S3 if >15 days -> 0%; else if >=48h -> 5%; else if >=24h -> 10%; else -> 20%
S4 round fee by 0.5 yuan rule
S5 refund = price - rounded fee
S6 output channel, fee, refund amount
```

场景矩阵：本矩阵先列业务分支，再展开成测试用例，避免只按时间随便挑几条。

场景类型	覆盖点	说明
时间边界	15 天、48 小时、24 小时、2 小时	这些点最容易因为大于/大于等于写错而出错
办理渠道	未打印且不少于 2 小时、已打印、不足 2 小时	渠道判断和手续费比例不是同一个判断
费用取整	小于 2.5 角、2.5 到 7.5 角、7.5 角以上	需要额外票价才能把取整边界测出来

编号	时间与票价	纸质票与渠道	场景说明	预期结果
S01	2026-06-14 07:51；票价 214.50	未打印；网上可退	大于 15 天	手续费 0.00，退款 214.50
S02	2026-06-15 07:52；票价 214.50	未打印；网上可退	正好 15 天，按 48 小时以上处理	手续费 10.50，退款 204.00
S03	2026-06-20 09:00；票价 214.50	未打印；网上可退	48 小时以上且不超过 15 天	手续费 10.50，退款 204.00
S04	2026-06-28 07:52；票价 214.50	未打印；网上可退	正好 48 小时	手续费 10.50，退款 204.00
S05	2026-06-28 08:10；票价 214.50	未打印；网上可退	不足 48 小时且超过 24 小时	手续费 21.50，退款 193.00
S06	2026-06-29 07:52；票价 214.50	未打印；网上可退	正好 24 小时，按 24 小时以上处理	手续费 21.50，退款 193.00

编号	时间与票价	纸质票与渠道	场景说明	预期结果
S07	2026-06-29 22:00；票价 214.50	未打印；网上可退	不足 24 小时但超过 2 小时	手续费 43.00，退款 171.50
S08	2026-06-30 05:52；票价 214.50	未打印；网上可退	正好 2 小时	手续费 43.00，退款 171.50
S09	2026-06-30 06:10；票价 214.50	未打印；窗口办理	不足 2 小时	手续费 43.00，退款 171.50
S10	2026-06-20 10:00；票价 214.50	已打印；窗口办理	时间满足网上退票但已取票	手续费 10.50，退款 204.00
S11	2026-06-20 10:30；票价 101.00	未打印；网上可退	补充票价：尾数小于 2.5 角	5% 为 5.05，取整 5.00，退款 96.00
S12	2026-06-20 11:00；票价 105.00	未打印；网上可退	补充票价：尾数 2.5 角到 7.5 角	5% 为 5.25，取整 5.50，退款 99.50
S13	2026-06-20 11:30；票价 115.50	未打印；网上可退	补充票价：尾数 7.5 角以上	5% 为 5.775，取整 6.00，退款 109.50

这组用例按本人对退票业务的理解独立设计。前十条主要卡时间和渠道边界，后三条专门卡手续费尾数取整，避免把金额规则只测成“按比例乘一下”。

七、 实验收获

后台类代码最需要测试的是保护条件。封禁用户如果没有最后管理员保护，系统可能把自己锁死；举报如果不防重复和自举报，管理队列会很乱；通知如果把他人的通知返回成“无权访问”，反而可能泄露通知存在。练习路径法时，我觉得这些分支比成功路径更有价值，因为它们是系统保持可管理和可恢复的底线。

这次练习让我更明显地感觉到，白盒测试不是看页面能不能点，而是要把代码里的判断条件拆出来。控制流图虽然画起来有点麻烦，但它能逼着我检查“是不是只测了成

功分支”。像登录限流、重复签到、库存不足这类路径，如果不画出来，很容易在写用例时被忽略。

JUnit 自动化对回归也很有帮助。手工看一遍代码只能说明当时理解了逻辑，测试方法保留下来以后，后面改服务层、改 DTO 或改仓储接口时，还能重新跑一遍确认分支没有被改坏。黑盒退票作业则提醒我，业务规则不一定难在流程长，而是难在边界：正好 48 小时、正好 2 小时、手续费尾数取整，这些点比普通样例更容易暴露问题。